Billion-files File Systems (BfFS): A Comparison

Sohail Shaikh
Department of Computer Science
George Mason University, Fairfax, Virginia
mshaikh5@gmu.edu

***Abstract*—As the volume of data being produced is increasing at an exponential rate that needs to be processed quickly, it is reasonable that the data needs to be available very close to the compute devices to reduce transfer latency. Due to this need, local filesystems are getting close attention to understand their inner workings, performance, and more importantly their limitations. This study analyzes few popular Linux filesystems: EXT4, XFS, BtrFS, ZFS, and F2FS by creating, storing, and then reading back one billion files from the local filesystem. The study also captured and analyzed read/write throughput, storage blocks usage, disk space utilization and overheads, and other metrics useful for system designers and integrators. Furthermore, the study explored other side effects such as filesystem performance degradation during and after these large numbers of files and folders are created.**

***Index Terms*—Linux, filesystem, EXT4, XFS, BtrFS, F2FS, ZFS, performance, statistics**

I. Introduction

The local filesystem, where the operating system and applications reside, forms the core of many systems. As local hard drives scale up, the number of objects stored in each node grows. This study is to investigate the hypothesis that there are likely some limitations and performance impacts of storing and accessing large numbers of files (e.g., a billion files) of varying sizes on popular local filesystems such as EXT4, XFS, BtrFS, F2FS, ZFS, etc. The study intends to evaluate different local filesystems available for the Linux operating system, their pros and cons, their ability to store a billion files and directories, and their suitability for the purpose under specific workloads on different types of storage media such as SSD and HDD.

Linux EXT4, the 3rd revision of the EXT filesystem has been around since the mid-2000 and has performed very well as a general-purpose and stable local filesystem for popular Linux distributions. However, as the data volume has been increasing rapidly with large amounts of data being placed closer to the compute devices to improve the performance of data-intensive computations and avoid data transfers over the network, the importance of local filesystem performance and their limitation has increased rapidly. The core idea of this study is to answer two basic questions supported by analysis: a) Can these filesystems handle one billion or more files and folders? and b) does the performance deteriorates as the number of files and folders are added? If unable to store a billion files, analyze and determine if these limits can be overcome.

EXT4 is currently the default filesystem for most popular Linux distributions. EXT2 and earlier filesystems were prone to catastrophic corruption when the system loses power during a write operation. EXT3 solved this problem by journaling, a 2-phase writing process of metadata and the data. EXT3 was limited to a 2TB file size and a maximum 16TB filesystem size. EXT4 increased this limitation to 16TB file size and 1EB filesystem size. There are other improvements in EXT4 such as different types of block allocations, extents, and an unlimited number of subdirectories using HTree indices [16], the checksum for journals, nanosecond timestamp, and online defragmentation [7].

XFS is a high-performance 64-bit journaling filesystem and was ported to the Linux kernel in 2001. XFS excels in the execution of parallel input/output (I/O) operations due to its design [4]. XFS design enables extreme scalability of I/O threads, filesystem bandwidth, size of files, and the filesystem itself when spanning multiple physical storage devices. XFS ensures the consistency of data by employing metadata journaling and supporting write barriers. Space allocation is performed via extents with data structures stored in B+ trees, improving the overall performance of the file system, especially when handling large files.

BtrFS is a modern modified Btree-based Copy-on-Write (CoW) filesystem where files are not replaced but a new copy with updates are created. It is aimed at implementing advanced features while also focusing on fault tolerance, repair, and easy administration. Its main features and benefits are snapshots, RAID, and self-healing. 16EB file size, dynamic inode allocation, SSD awareness, automatic defragmentation, and scrubbing [6].

ZFS is another filesystem developed by Sun Microsystem and acquired by Oracle, combines the roles of volume manager and a file system intended for critical systems where data loss is not acceptable. ZFS is intended for large infrastructures where large number of storage devices are deployed. ZFS requires appropriate configuration to support these storage devices.

F2FS (Flash-Friendly File System) is a filesystem specifically intended for NAND-based flash memory devices equipped with Flash Translation Layer (FTL). It is supported by Linux kernel 3.8 and above and supports compression as well as encryption natively. F2FS was designed around the idea of a log-structured filesystem approach, which is adapted to newer forms of storage and overcame issues such as the snowball effect of wandering trees and high cleaning overhead. F2FS has a weak *fsck* that can lead to data loss in case of a sudden power loss [8].

II. METHODS

To compare target filesystems, they needed to be exercised thoroughly and compared against a baseline. The baseline is established by creating 10 million files of different sizes that are *normally* distributed between 1KB and 10KB, and stored evenly among 100 root folders. To establish a baseline, the first step is to set up the environment for these experiments. Due to several factors including the introduction of unpredictable internet and ISP-induced latencies, and response time from the cloud components, it was decided to avoid the cloud altogether and rely on the on-prem server hardware for experiments. The specification of the on-prem HP Z820 workstation is: 2x Xeon E5-2670 2.6GHz CPUs with 16 cores/32 threads total, 256GB RAM, dual NICs connected to two independent subnets, one 1TB SSD for Linux Ubuntu 22.04, a 2TB Samsung 870 EVO SATA SSD [12] with 550MB/sec read/write performance for applications, and a 14TB Seagate IronWolf™ HDD with 256MB cache and a 210MB/s sustained transfer rate [11]. In addition, C/C++ development tools, Java OpenJDK 11, Visual Studio, and other tools were installed such as filebench, fio, vmstat, iotop, dstat, atop, visualjvm, and considered using bcc [17]. The goal is to capture IOPS, read and write performance, and I/O latency of reading and writing files. To exercise these filesystems with files and folders reads and writes, an application was developed. The operating system was not fine-tuned in any way deliberately to make this study reproducible and useful for others.

A. Application

After compiling *filebench* and making several attempts to test and failed at around 10 million files mark due to crashes, it became evident that it is time to write some purpose-built tools. Initially, Java environment is used for its flexibility and consistent performance to develop the benchmarking and evaluation application [2]. To ensure that application overheads are consistent regardless of the number of files to be created or read, simple programming constructs were used. In addition, to eliminate any performance degradation due to introduction of a JVM, the application is also (re)developed in C language using the lowest level Linux file system calls [19]. This paper uses performance metrics from C application. The high-level pseudo code for folders and files creation and verification are shown in Algorithm 1.

The application consists of two parts: a *Creator* and a *Reader*, routines that create folders, subfolders and files and read folders, subfolders and files, respectively. To accommodate 1 billion files with a planned file size range of 1KB to 10KB, a 14TB HDD was used. The rationale behind the file size range is that a typical disk block size is 4KB so a file may use anywhere from one and up to three 4KB blocks [3]. During file creation, each file is added with random binary data and is appended with an 8-byte CRC-32 checksum generated on-the-fly so that the *Reader* can verify the file's content during reading.

The Creator routine also captures file sizes and individual write throughput distribution during the runs. At the end of the run, it summarizes captured metrics which are then fed into an Excel spreadsheet for further analysis. The Reader performs the reads, iterating thru the folders and files created by the Creator earlier. The Reader reads each file's content, extracts the last 8 bytes of the content, the CRC-32 checksum, computes the CRC-32 checksum of the remaining content, and compares the two to ensure file's integrity. The application counts the number of failed checksum and reports the file count discrepancy for the folder as well as for the entire run at the end. Following is the files/folders schedule that is used for testing these file systems: $Files = Folders{*}Subfolders{*}$ 100K.

Algorithm 1 Folders/Files create/write/read algorithm

```
Files ← 10M or 100M or 1B files
Folders ← 100
Subfolders ← computed from Files
for i = 1 to Folders do
    Create folder
    for j = 1 to Subfolders do
        Create subfolder
        for k = 1 to 100K do
            Create file of normal random data size with crc-32
        end for
    end for
    Statistics ← collect folder/file statistics
end for
Statistics ← collect storage statistics
for i = 1 to Folders do
    Search folder
    for j = 1 to Subfolders do
        Search subfolder
        for k = 1 to 100K do
            Read file and verify content using crc-32
        end for
    end for
    Statistics ← collect folder/file statistics
end for
Process collected statistics for analysis
```

TABLE I: Files/Folders Read/Write Schedule

Files	Folders	Subfolders/Folder	Files/Subfolder
10M	100	1	100,000
100M	100	10	100,000
1B	100	100	100,000

In addition to reading and writing files and folders, the application captures metrics for *macrobenchmarking* and *microbenchmarking*. Macrobenchmarking captures read/write performance metrics for the individual folders and files contained in them as well as the entire run at the end whereas microbenchmarking focuses on individual file read/write performance and distribution (min, ave., max). It is important to note that captured timings are in microseconds.

B. Linux Filesystem

Focusing only on Linux operating system for this study, Linux file system is designed around several abstract layers built on top of each other for ease of interfacing and to isolate the applications from unnecessary complexities. Applications access block devices, such as hard disks through system calls (e.g., *open()*, *read()*, *write()*) to the Virtual File System (VFS). VFS is the software layer in the kernel that provides the filesystem interface to the user space programs[5] and interfaces with one or more installed file systems such as EXT4, XFS, and others. Between the hardware and file system, there exist two more layers: the page cache and the block layer.

Linux page cache layer is the main disk cache and improves file system performance by caching data to and from the disk. The kernel first looks in the cache to see if the data is available and only accesses the disk if the data is not found in the cache. Once the data is read from the disk, the page cache is updated. In the case of a write operation, data is checked if it is already in the cache, if not a new entry is added but the data is not written immediately to the disk to allow accumulating more data before committing to the disk, reducing the I/O overheads.

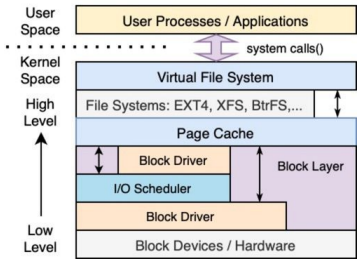


Fig. 1: Linux VFS Architecture

This delayed write to the disk is handled by the I/O scheduler in the block layer (which improved write speed). Block drivers hide the complexity of the underlying storage hardware e.g., hard disks (HDD), solid-state disks (SSD), optical disks, etc. In the context of this paper, there are a few other important aspects of the filesystem implemented in Linux VFS that are useful to understand for this study and explained in the following sections, e.g., directory entry cache, inode, and file object. Read/Write latency for the file system is described as follows where each *layer* contributes to some amount of latency: $Latency_{read/write} = Latency_{application} + Latency_{VFS} + Latency_{filesystem} + Latency_{driver} + Latency_{diskIO}$.

Application latency is largely dependent on the programming language and constructs to access the filesystem, and any high-level abstractions used to make the filesystem transparent to the programmer. Linux Virtual File System (VFS) latency is related to abstracting the filesystem's internal working from the high-level system utilities as well as the applications so that different filesystems look the same to the applications [10].

However, the latency imposed by the application is assumed to be constant or with minimum variations between runs. VFS also imposes a negligible overhead as compared to the amount of time spent in other layers. Since the application, block driver, and block device layers are constant therefore the only layer left is the filesystem which is the layer this study tries to capture and analyze. The study has analyzed the portion of run time that is consumed by the application, other software layers, and by the hard disk I/O.

1) Directory Entry Cache: File-related system calls such as *open()* and *create()* require *path* as an argument. This argument is used by the VFS to search for the directory entry in a lookup cache, called Directory Entry Cache (*dcache*) and it provides a view into the entire filespace. Due to the constraint of limited physical memory, entries (*dentries*) are sometimes created on the fly using inode information.

2) Inode Cache and Inode: Inode cache (*icache*) go together with *dcache*, in a master-slave configuration. If there is a *dentry*, there is an entry in *icache*. The reason *icache* exists in the first place is that the *inode* gets updated frequently while the file is open but the inode is saved on the disk unlike *dcache*. An inode or index node exists for one per object in the filesystem for all object types (files, directories, etc.) It is a structure and holds not only the inode number but also the file size, owner's user id, and group id, access and creation times, etc.

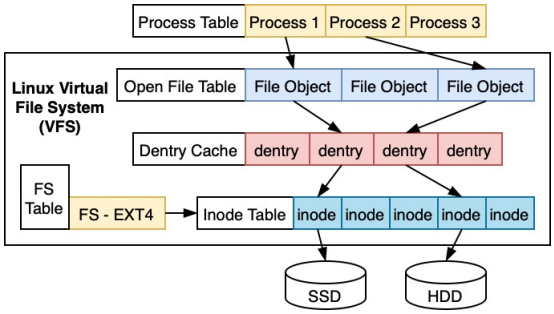


Fig. 2: EXT4 Internal Structures

3) File Object: For each opened file in a Linux system, a file object exists and contains information such as its path, *dentry* entry, file operations, flags, permission, mode, and other metadata[10]. Each file is assigned a file descriptor (FD) which points to the file object and is used for file operations. Linux reserves the first three FDs for standard input, output, and error and imposes a configurable 1024 FDs per-process limit.

C. Discovery Approach

To determine the read/write behavior of the target filesystems namely: EXT4 (baseline), XFS, BtrFS, ZFS and F2FS, several tools were explored but nothing satisfied the goals of this study. To overcome this roadblock, an application is developed in Java and later in C language which creates files and folders and captures performance metrics for further analysis. The targeted metrics are described below for writes and reads.

1) Baseline Metrics - $EXT4_{10M}$ filesystem: As the first step, a baseline needs to be established for comparison before conducting any further analysis. EXT4, which comes out of the box with most Linux distributions, is used as the baseline

filesystem that allows further analysis and comparison of targeted filesystems to support one billion files. Application program Creator and Reader ran on the 14TB HDD and produced the results listed in Table II.

TABLE II: $EXT4_{10M}$ Baseline Disk Metrics

Parameter	Before	Used	After
Inodes	1200005235	10000200	1190005035
Blocks	3171666459	19270745	3152395714
Disk size	1.2991×10^{13}	78932971520	1.2912×10^{13}

2) File Writes: The application provides the capability to create folders and then create files between the sizes of 1K and 10K following a normal distribution curve, using Box-Muller transform [18], that produced a consistent file sizes with the median falling around 5500 bytes with std. dev. of 1024 bytes. Reason to chose this range of file sizes is to ensure that one billion files fit in the 14TB hard disk as well at least 66% of the file sizes are over 4KB block size occupying at least two blocks (except in the case of ZFS which uses 128KB blocks).

The file size distribution pattern is shown in the Figure 3. The tool also captured additional information to assess the performance. Individual files write time is aggregated for each folder and used for cumulative write time for the entire run. It is shown in Table III as the Total Write Time (TWT) in μsec. Captured individual file write time is also used to determine the min, max and average file write times. Total bytes written on the disk are captured to determine the disk utilization and overheads of adding the files. The tool captures an exact number of bytes per file and per folder which are added to get the cumulative size written to the disk for each run. After each run, inode count, blocks used, and increase in the disk space utilization are captured to determine the overheads.

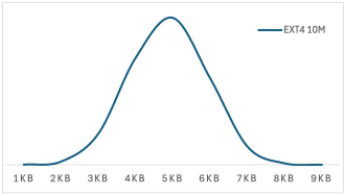


Fig. 3: File Size Distribution

3) File Reads: In order to understand the differences between write performance and read performance, we have to consider that during the write operation the cache is used therefore the system calls return quicker than in the case of reads since the application reads all the bytes in the memory, computes CRC-32 check, and compare with what is provided in the file before proceeding to the next file. File systems provide different read performance considering number of files generated earlier. For this comparison, the paper consider the average write and average read statistics. The captured average write performance is between 10$\tilde{1}$5 μsec whereas read performances varies between 5 to 200 μsec. Except in the case of ZFS, the difference between write speed and read speed of the same files was not too great, e.g., on average $EXT4_{100M}$ write is 24 μsec whereas read is 62 μsec which is 3 times slower. However, for ZFS_{100M} write is 16 μsec but the average reads are 201 μsec, a 12 times slower performance.

4) Disk Utilization Overheads: Disk utilization overhead is another aspect of the file creation process where the disk space used is greater than the bytes added, which is calculated by *((Disk Space Used - Bytes Added) / Disk Space Used) * 100*, and has been observed to be anywhere from 30% to 60% for different filesystems. However, except in one case (i.e., XFS) number of files and folders did not affect this overhead because of its handling of folders and files metadata dynamically.

Though total folder creation time is relatively small, the tool captures these metrics to ensure it is not high enough to skew the analysis, especially for larger runs e.g. 10M files and above. Per folder write throughput which is determined by individual file write time divided by the number of files per folder. It is used to analyze performance degradation as the number of folders and files are added to the filesystem.

Each file's write speed is captured, sorted, and saved in a bucket to determine the distribution pattern as shown in Figure 4. For example, majority of EXT4 file write speeds fall in two buckets: 10-15μs and 15-20μs whereas XFS write speed fall precisely 15μs regardless of number of files. Each of the file systems listed below requires a slightly different configuration to ensure it can accommodate one billion or more files and directories.

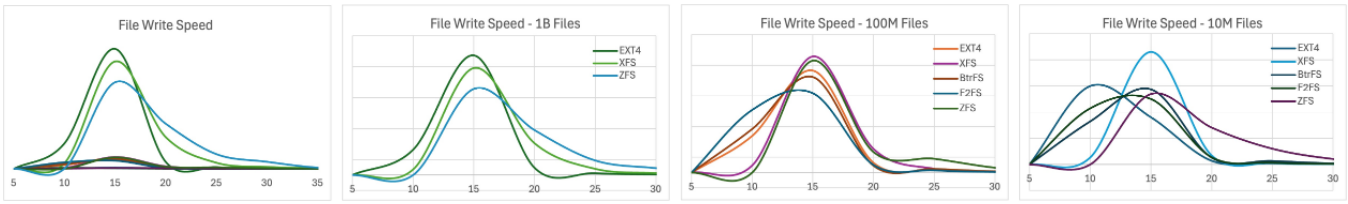
EXT4 is the general-purpose filesystem for a majority of Linux distributions and gets installed as the default. However, it is unable to accommodate one billion files by default due to a small number of inodes available, typically set at 240 million files or folders, created during installation. To increase the inodes, the EXT4 filesystem needs to be reinstalled on a device with a custom configuration using the *mkfs* command. The option -b 4096 creates block of size 4KB each resulting in about 1.2 billion inodes on a 14TB drive: *mkfs.ext4 -N 1200005248 -b 4096 /dev/xxx*

XFS creates inodes dynamically as the files are being created but it requires that enough space is allocated to the inodes data structures on the disk, therefore XFS can handle one billion or more files and folders. This dynamic creation of inodes was observed when the progress bar momentarily pauses and then continues during the runs. For this purpose *maxpct* flag is used with the command that allocates a percentage of the disk to the inode structure e.g. 10% as shown in the command: *mkfs.xfs -i maxpct=10 -f /dev/xxx*

BtrFS does not use inodes the like other filesystems therefore it always shows inode=0 for *df* command. BtrFS has many other advanced features for a modern filesystem such as snapshots, fault tolerance, copy-on-write, support for huge size files, etc. The command to create BtrFS is: *mkfs.btrfs -f /dev/xxx*

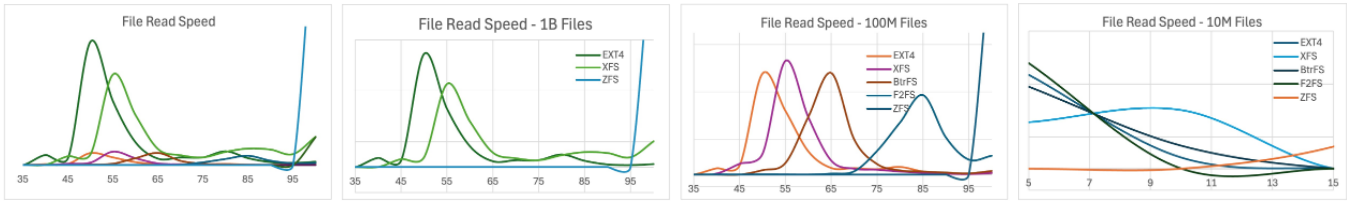
ZFS requires that a pool is created first and then it needs to be mounted. Only then it is available for write/read operations. A pool is created using: *zpool create -f zfspoolname /dev/xxx*.

F2FS is a specialized filesystem to support SSDs. Upon installation, F2FS established a segmented disk layout. It uses inodes to points to files and folders. The command used was:



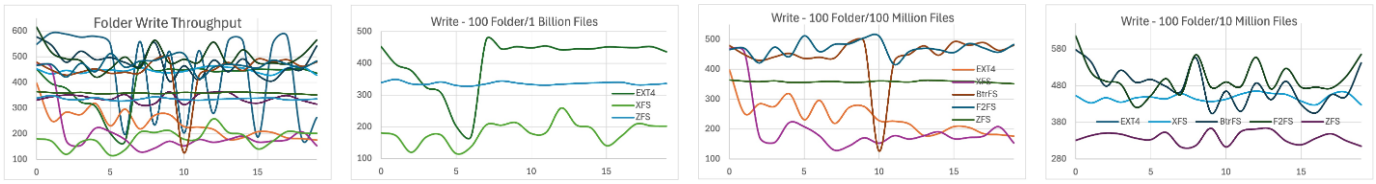
(a) All Files (b) 1 Billion Files (c) 100 Million Files (d) 10 Million Files

Fig. 4: File Write Speed Distribution (in μsec)



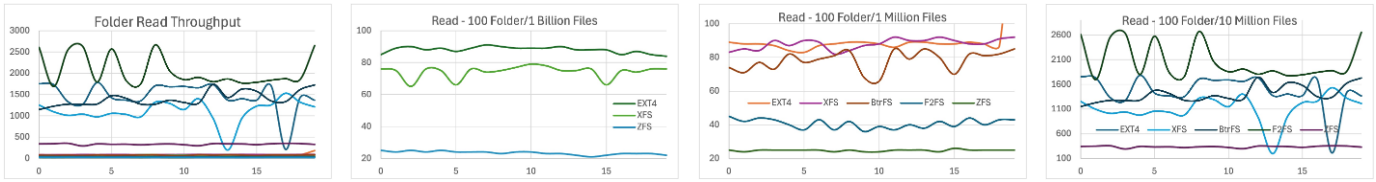
(a) All Files (b) 1 Billion Files (c) 100 Million Files (d) 10 Million Files

Fig. 5: File Read Speed Distribution (in μsec)



(a) All Files (b) 1 Billion Files (c) 100 Million Files (d) 10 Million Files

Fig. 6: Folder Write Throughput Distributed Over Time (μsec vs 20 buckets)



(a) All Files (b) 1 Billion Files (c) 100 Million Files (d) 10 Million Files

Fig. 7: Folder Read Throughput Distributed Over Time (μsec vs 20 buckets)

mkfs.f2fs -i -s 10 -z 10 -f /dev/xxx. This generated only 630 million inodes and unable to create 1 billion inodes so the test was restricted to 100 million files and corresponding folders.

The tables below captured performance statistics for writes and reads for the file systems listed above. The explanation of write performance numbers are as follows:

- $\text{FWT}_{(\text{min, ave, max})}$: File Write $\text{Time}_{(\text{min, ave, and max})}$ in μsec.
- WTh: Write Throughput in bytes per microsecond.
- TFWT: Total Folder Write Time is cumulative time to create (sub)folders in seconds.
- TfWT: Total File Write Time is time to create/write files in seconds.
- TWT: Total Write Time is the time to create folders, subfolders and files in seconds.
- FWs: Files written per second.
- BkWs: Blocks written per second. Typically 4KB for file systems but for ZFS it is 128KB.
- TFCWT: Total File Create for Write Time is the cumulative time to create files but does not include writing the data/bytes to the file.
- FCWT: Average time to create a files but not writing the bytes to the file.
- TByW: Total Bytes Written in gigabytes (GB).
- TBkW: Total 4KB or 128KB Blocks Written.
- DSU: Disk Space Used in gigabytes (GB).
- DSUO: Disk Space Utilization Overheads.
- Inodes: Inodes used to store folders, subfolders and files.

The explanation of read performance numbers are as follows:

- $\text{FRT}_{(\text{min, ave, max})}$: File Read $\text{Time}_{(\text{min, ave, and max})}$ in μsec.

- RTh: Read Throughput in bytes per microsecond.
- TFRT: Total Folder Read Time is cumulative time to search (sub)folders in seconds.
- TfWT: Total File Read Time is time to read files in seconds.
- TWT: Total Read Time is the time to search (sub)folders and files in seconds.
- FWs: Files read per second.
- BkWs: Blocks read per second. Typically 4KB for file systems but for ZFS it is 128KB.
- TFORT: Total File Open for Read Time is the cumulative time to open files but does not include reading the data/bytes from the file.
- FORT: Average time to open and read a files.
- TByR: Total Bytes Read in gigabytes (GB).
- TBkR: Total 4KB or 128KB Blocks Read.
- TRT: Total Run Time in seconds.
- CPUO: Computation Overheads is the total time taken by the run not including I/O.
- BkSize: Block size, typically 4KB except for ZFS's 128KB.

Total Runtim Time (TRT) can be loosely defined as Total Write Time (TFCWT + TWT), Total Read Time (TFORT + TRT) + processing overheads.

III. Discussion

The application generates folders, subfolders and files as per schedule in Table I. The files created with random data with normal distribution as shown in Figure 3 appended with a CRC-32 checksum computed using the random data generated for the file. Once all the files are generated, they are read back according to the same schedule. The file data is verified using CRC-32 read from the file and a generated CRC-32 checksum using the data from the file. Using EXT4 file system with 10 million files spread between 100 root folders and 1 subfolder each as the baseline to compare other file systems. The reason is to minimize the differences between each run and the configuration. All the runs were carefully conducted. Linux Ubuntu 22.04 was out-of-the-box version with no configuration changes except regular system updates, re-creating the file system on the 14TB drive, rebooting the server between switching to different file system as well as in between runs when generating 1 billion files as these runs some times lasted 36 to 48 hours. Most of initial testing was conducted using EXT4 with 10 million files. Comparison analysis is performed between the baseline EXT4 and other file systems.

File write and read performance provided a glimpse of each file system and how they behave with different number of folders, subfolders, and files as each consumes an inode. As captured, the application is I/O-intensive, which means it spent majority of time waiting for disk I/O, i.e., writing or reading or searching the files. CPU overhead captured the numbers to show the percentage of the time system was *not* performing I/O. Starting with an initial assumption about worse write performance in comparison with read performance which turned out to be false because of Linux's built-in write caching mechanism. For example, EXT4 1B average file write time is only 14 μsec but the average read time is 62 μsec, about x3 times slower. ZFS 1B run is even worse in this regard since its average file write time is 16 μsec but the average file read time is 218 μsec, about x12 times slower. File read performance numbers provided a consistently interesting pattern as the graph shows. All small runs i.e., 10M files, for all filesystems, provided slower read speeds than write times.

One of the questions that needed investigation was: Is there any performance degradation as the number of files in the filesystem increases? To answer this question, Creator code computes and uses folder performance over the entire run. Due to large number of folders it was difficult to show the trend over all folders, therefore it was decided to take 20 samples during the entire run spaced evenly.

The graphs in Figures 6a and 7a shows all filesystems with different runs. These graphs are further categorized by number of files generated or read to show different patterns at different times of the runs. The purpose is show the long term trends especially during the folders, subfolders and files creation. In some cases, write performance recovered and in some cases continue to deteriorate. Starting with EXT4, both read and write performance was consistent for all three runs though EXT4 shows momentary deterioration initially during 1 billion files run which was not reproducible so it was recorded as a possible anomaly. EXT4 gave relatively consistent write and read performances as well as long term performance degradation trends among the set of file system explored.

XFS turned out to be about the same in performance and long term performance deterioration trend when compared with EXT4 (EXT4/XFS: WR:15/12, 24/28, 14/31 RD:10/5, 62/62, 62/73). In 1 billion folder read/write performance XFS was slower than EXT4 (see Figures 6b and 7b).

We need to point out that there was an initial performance degradation in 100 million files run for both EXT4 and XFS which seems to be stabilized after create 25 folders and files. This degradation was not observed in 1 billion or 10 million files runs.

XFS turned out to be about the same in performance and long term performance deterioration trend when compared with EXT4 (EXT4/XFS: WR:15/12, 24/28, 14/31 RD:10/5, 62/62, 62/73).

In 1 billion folder read/write performance XFS was slower than EXT4 (see Figures 6b and 7b). We need to point out that there was an initial performance degradation in 100 million files run for both EXT4 and XFS which seems to be stabilized after create 25 folders and files. This degradation was not observed in 1 billion or 10 million files runs.

TABLE III: File Write Performance Metrics

Filesystem	$FWT_{min,ave,max}$	WTh	TFWT	TfWT	TWT	FWs	BkWs	TFCWT	FCWT	TByW	TBkW	DSU	DSUO	Inodes
$EXT4_{10M_{BL}}$	5, 15, 2.53×10^6	348	0.02	157	157	63	121	253	25	54.99	19.18	78.93	30%	10000200
$EXT4_{100M}$	5, 24, 2.50×10^6	224	0.12	2452	2452	41	78	3103	31	549.95	191.82	789.29	30%	100001100
$EXT4_{1B}$	5, 14, 2.23×10^6	382	18.33	14387	14405	70	133	24762	24	5507.59	1919.61	7898.75	30%	1000010100
XFS_{10M}	7, 12, 0.22×10^6	442	0.09	124	124	80	154	224	22	54.99	19.18	138.34	60%	10000200
XFS_{100M}	7, 28, 0.76×10^6	191	0.31	2871	2872	35	66	2425	24	549.96	191.83	903.31	39%	100001100
XFS_{1B}	7, 31, 0.72×10^6	176	8.07	31208	31216	32	61	26242	26	5499.58	1918.25	8432.62	35%	1000010100
$BtrFS_{10M}$	6, 11, 0.14×10^6	464	0.01	118	118	85	162	220	22	54.99	19.18	85.23	35%	0
$BtrFS_{100M}$	6, 12, 30.73×10^6	445	0.07	1234	1234	81	155	4968	49	549.93	191.82	911.62	40%	0
$F2FS_{10M}$	4, 11, 0.19×10^6	495	0.01	110	110	90	172	1131	113	55.00	19.18	118.99	54%	10000400
$F2FS_{100M}$	4, 11, 0.39×10^6	471	0.05	1166	1166	86	164	11969	119	549.97	191.83	1189.78	54%	100003100
ZFS_{10M}	11, 16, 0.05×10^6	337	0.02	169	163	61	61	394	39	55.00	10.00	82.33	33%	10000200
ZFS_{100M}	11, 15, 0.09×10^6	358	5.92	1535	1541	65	65	3901	39	549.94	100.00	823.22	33%	100001100
ZFS_{1B}	11, 16, 0.13×10^6	337	13.76	16316	16330	61	61	40887	40	5499.59	1000.00	8232.76	33%	1000010100

All performance numbers are in microseconds. Table is divided in rows and columns: rows are for file system and number of files generated, whereas columns capture particular metrics. Each column is explained in detail the paper. EXT4 10M files is the baseline ($EXT4_{10M_{BL}}$). BtrFS file system does not use inodes therefore it is shown with 0 inodes. Unable to read back 1 billion files for BtrFS file system due to exponentially increasing read back times therefore it is not included in the write or read tables. Please note that folders are also represented by inodes.

TABLE IV: File Read Performance Metrics

Filesystem	$FRT_{min,ave,max}$	RTh	TFRT	TfRT	TRT	FRs	BkRs	TFORT	FORT	TByR	TBkR	TRT	CPUO	BkSize
$EXT4_{10M_{BL}}$	2, 10, 4.12×10^6	505	0.00	108	108	92	176	23	2	54.99	19.18	683.66	21%	4096
$EXT4_{100M}$	2, 62, 1.73×10^6	88	0.05	6226	6226	16	30	1688	16	549.95	191.82	15049.99	10%	4096
$EXT4_{1B}$	2, 62, 0.95×10^6	88	185.42	62200	62386	16	30	20181	20	5507.59	1919.61	136914.58	11%	4096
XFS_{10M}	2, 5, 2.16×10^6	1028	0.00	53	53	187	358	23	2	54.99	19.18	571.45	26%	4096
XFS_{100M}	3, 62, 1.17×10^6	87	0.09	6285	6285	16	30	1351	13	549.96	191.83	14560.15	11%	4096
XFS_{1B}	37, 73, 1.17×10^6	74	253.65	73852	74105	14	25	39588	39	5499.58	1918.25	188036.72	9%	4096
$BtrFS_{10M}$	2, 4, 0.0×10^6	1349	0.00	40	40	245	470	24	2	54.99	19.18	553.70	27%	4096
$BtrFS_{100M}$	2, 70, 0.62×10^6	77	0.08	7071	7071	14	27	1759	17	549.93	191.82	16674.51	10%	4096
$F2FS_{10M}$	1, 2, 0.01×10^6	2038	0.00	26	26	371	710	23	2	55.00	19.18	1436.08	10%	4096
$F2FS_{100M}$	2, 131, 0.66×10^6	41	1.36	13163	13164	8	14	20780	207	549.97	191.83	48771.11	3%	4096
ZFS_{10M}	11, 16, 0.06×10^6	332	0.00	165	165	61	60	30	3	55.00	10.00	912.24	17%	131072
ZFS_{100M}	83, 217, 0.65×10^6	25	2.91	21707	21710	5	4	3961	39	549.94	100.00	32981.95	6%	131072
ZFS_{1B}	73, 230, 0.83×10^6	23	10.52	230447	230457	4	4	41671	41	5499.59	1000.00	348312.94	5%	131072

All performance numbers are in microseconds. Table is divided in rows and columns: rows are for file system and number of files read, whereas columns captures a particular metrics. Each column is explained in detail the paper. Right most three columns are included though they are common for write and read operation. Total Run Time (TRT) is the total run time of the application against a target file system. CPUO is the CPU bound processing overheads that excludes any I/O. BlkSize is the block size used by the file system.

To capture the long term trend to assess performance deterioration, we captured write and read performance of every 5th folder created and read back and stored these numbers in 20 buckets. BtrFS testing took more time than all others file system testing combined because its read performance deteriorated to a point that 1 billion files run never got completed even after running for 72 hours straight.

The write portion of the runs completed but reads were too slow to over 60 minutes and gradually increasing just to read one folder so runs were killed, rebooted, re-installed file system and still failed. BtrFS file write was comparable to EXT4 but the file read performance was lightly lower (around 65 μsec). We did not observe any long term write or read performance degradation for BtrFS for 100 million and 10 million files runs.

F2FS, though meant for flash drives, was tested on 14TB HDD to be consistent with other file systems under test. It was not expected we could create 1 billion files but we tested F2FS anyway by increasing the inodes. We were unable to increase beyond 600 million so restricted F2FS to 100 million files maximum. F2FS was slightly better than EXT4 in file write performance but in file read performance F2FS was slower than EXT4 in 100 million files runs. In folder write and read long term trend, F2F2 was stable in both 100 million and 10 million files runs.

ZFS was different in terms of its installation. Once installed it behaved like any other system, i.e., no changes to the application code required. ZFS file write performance was very consistent (16/15/16 μsec) but file read performance was much higher (16/217/230 μsec). ZFS runs took much higher clock time than other file systems. For 1 billion files write and read back it took 330 thousand seconds, about 92 hours, to complete.

CPU overheads are defined as time the code spend not performing any I/O, such as calculating statistics, or other housekeeping work. The decision to switch to pure C from Java was deliberate in order to avoid any overhead imposed by JVM or Java programming constructs. The number captured showed a consistent pattern: 20% for 10 million files runs, 10% for 100 million files runs, and 10% for 1 billion files runs. CPU overheads were computed using Total Run Time (TRT), Total Write Time (TWT), Total File Create for Write Time (TFCWT), Total Read Time (TRT), and Total File Open for Read Time (TFORT). CPUO = TRT - (TWT + TFCWT + TRT + TFORT) / TRT

IV. Conclusion

This study investigated and compared several popular Linux filesystems: EXT4, XFS, BtrFS, F2FS, and ZFS for their ability to create and manage one billion files and measure file creation and reading timing and any performance degradation during and after creating the files. Except for F2FS, all filesystems were able to handle one billion files after increasing the inodes and re-mounting the filesystem i.e., no other operating system onfiguration changes were made. BtrFS was tested with 1 billion files but reading these files were too slow to capture the final performance metrics. To exercise these file systems and capture desired metrics, a tool is developed (initially in Java) in C programming language. The second decision was made to use an on-prem server rather than using the cloud to avoid unnecessary round-trip delays.

The decision to pick popular Linux filesystems was based on a couple of factors: their availability under most Linux distributions, non-clustered, and at least one, F2FS, supports SSDs natively. EXT4 comes with many Linux distributions by default therefore it was chosen to establish the baseline for performance benchmarking. XFS, BtrFS and ZFS were chosen because they were comparable in features with EXT4 for this paper.

All three filesystems, except F2FS, were able to handle one billion files. Once they ran successfully, and essential metrics was captured, these large tests were not repeated because they took days to complete. One of the fundamental limitations that were discovered was the default number of inodes. EXT4 needed to be configured and reinstalled whereas XFS, BtrFS and ZFS handle inode creation automatically. However, XFS requires that when installing the filesystem, a certain percentage of storage must be dedicated to inode data structures. EXT4 is a good choice if a large number of small files need to be stored on the filesystem. XFS has better block-based performance if performance is the primary requirement.

Original questions that were asked in the beginning are partially answered–ability to create one billion files on selected filesystems except for F2FS due to the lack of needed inodes. In addition, the study was able to answer about performance degradation. Except for EXT4, other filesystems show some signs of performance deterioration as the number of files increased, BtrFS was the worse in this regard. In addition, disk utilization overhead were observed for all four filesystems, XFS has the worse overheads followed by F2FS, useful for storage estimation by system designers.

The study captured extensive amounts of data for all four filesystems including individual read and write speeds as well as cumulative reads and writes performance, read, and write throughput, the number of files written and read per second, disk block utilization, and disk utilization overheads. Further work can be conducted to measure performance using real-world databases on these filesystems with high data volume and high transaction rates. Additional statistical analysis can be performed by working with the raw data captured to establish a statistical basis for the findings using techniques to include Confidence Intervals, Paired Observations, Linear Regression, Hypothesis Testing, and Analysis of Variance (ANOVA). We hope that study is useful for system designers in selecting the right file system for thier purpose.

Acknowledgment

I would like to acknowledge my professor Dr. Yue Cheng (mrz7dp@virginia.edu) who provided guidance and timely advice to conduct this study and in the preparation of this paper.

REFERENCES

[1] V. Tarasov, S. Bhanage, E. Zadok, and M. Seltzer. (2011). Benchmarking file system benchmarking: it *IS* rocket science. Workshop on Hot Topics in Operating Systems, 9.

[2] S. He, X.-H. Sun, and Y. Yin, "BPS: A Performance Metric of I/O System," in 2013 IEEE International Symposium on Parallel & Distributed Processing, Workshops and Phd Forum, 2013, pp. 1954–1962.

[3] "XFS Wikipedia," https://en.wikipedia.org/wiki/XFS (accessed Apr. 29, 2022).

[4] "BtrFS Wikipedia," https://en.wikipedia.org/wiki/BtrFS (accessed Apr. 29, 2022).

[5] "BtrFS Kernel," https://btrfs.wiki.kernel.org/index.php/Main_Page (accessed Apr. 29, 2022).

[6] J. Salter, "Understanding Linux filesystems: EXT4 and beyond," https://opensource.com/article/18/4/ext4-filesystem (accessed Apr. 29, 2022).

[7] "F2FS ArchLinux," https://wiki.archlinux.org/title/F2FS (accessed Apr. 29, 2022).

[8] R. Gooch "Overview of the Linux Virtual File System," https://www.kernel.org/doc/html/latest/filesystems/vfs.html, 1999, (accessed Apr. 29, 2022).

[9] M. Senofsky and P. Dietl "Introduction to the Linux Virtual Filesystem (VFS)," https://www.starlab.io/blog/introduction-to-the-linux-virtual- filesystem-vfs-part-i-a-high-level-tour. 2020, (accessed Apr. 29, 2022).

[10] Seagate Technology "Seagate 6TB-14TB IronWolf NAS HDD Technical Specifications," https://www.seagate.com/www-content/datasheets/pdfs/ironwolf-14tb-DS1904-10-1807US-en_US.pdf. 2022, (accessed Apr. 29, 2022).

[11] Samsung "Samsung 2TB 870 EVO SSD Technical Specifications," https://www.samsung.com/us/computing/memory-storage/solid-state-drives/870-evo-sata-2-5-ssd-2tb-mz-77e2t0b-am/. 2022, (accessed Apr. 29, 2022).

[12] "Hard disk drive performance characteristics," https://en.wikipedia.org/wiki/Hard_disk_drive_performance_characteristics. Aug. 2022, (accessed Apr. 29, 2022).

[13] M. Jones "Anatomy of the Linux virtual file system switch," https://developer.ibm.com/tutorials/l-virtual-filesystem-switch/, Aug. 2009, (accessed Apr. 29, 2022).

[14] M. Larabel "Linux 5.14 SSD Benchmarks With Btrfs vs. EXT4 vs. F2FS vs. XFS," https://www.phoronix.com/scan.php?page=news_item&px=Linux-5.14-File-Systems, Aug. 2021 (accessed Apr. 29, 2022).

[15] D. Phillips "A Directory Index for Ext2," http://www.linuxshowcase-.org/2001/full_papers/phillips/phillips_html/index.html, Sep. 2021 (accessed Apr. 29, 2022).

[16] B. Gregg "Linux Performance," https://www.brendangregg.com/linux-perf.html (accessed Apr. 29, 2022).

[17] D. J. Lilja, Measuring Computer Performance. Cambridge: Cambridge University Press, 2000.

[18] "Box-Muller Transform," https://en.wikipedia.org/wiki/Box–Muller_transform (accessed June. 30, 2024).

[19] Shaikh, S. (2024). BfFS (Version 1.0) [Computer software]. https://github.com/sshaikh5/BfFS